Cryptographic Hash Functions

CS 161 Spring 2026 - Lecture 9



Last Time: Secret-Key Encryption and Modes of Operation

Important primitives

	key	input	output
○ Pseudorandom function	$F : \{0,1\}^k$ x	$\{0,1\}^n \rightarrow$	$\{0,1\}^m$
○ Block cipher	$E, D : \{0,1\}^k$ x	$\{0,1\}^n \rightarrow$	$\{0,1\}^n$

○ Security: For secret random k, outputs of $F(k, \cdot)$ are "as good as random"

$E(k, \cdot)$

Mode of operation (e.g., counter mode, CBC mode):

○ How to use a PRF/block cipher to encrypt

○ Encrypt arbitrary-length plaintexts, in an CPA-secure way

Nothing so far provides any integrity/authenticity guarantee!

Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem

Cryptographic Hash Functions



Textbook Chapter 7.1–7.3

Cryptographic Hash Function

Hash function $H : \{0,1\}^* \rightarrow \{0,1\}^n$

- An efficiently computable deterministic function
  ⇒ Same input gives same output every time
- Input: *Arbitrary*-length bitstring
- Output: n-bit hash – often use $n = 256$

Properties of cryptographic hash functions

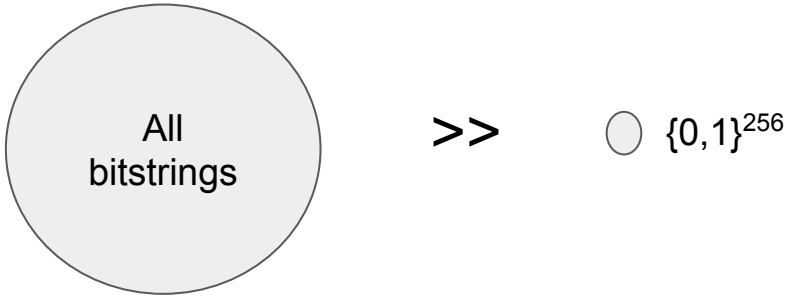
- Collision-resistance
- One-wayness (“preimage resistance”)
- …

Warning: Not all hash functions are cryptographic! (i.e., are one-way or collision resistant)

Definition: Collision resistant hash function (CRHF)

A hash function H is **collision resistant** if
no efficient adversary can find distinct x and x' such that $H(x) = H(x')$.

By pigeonhole principle there are **infinitely many** collisions in <u>every</u> hash function $H : \{0,1\}^* \rightarrow \{0,1\}^{256}$



If *H* is CRHF: Many collisions in *H* exist. They are just hard to find.

Why we use hash functions with 256-bit output

- **Birthday Paradox:** If you pick $2^{n/2}$ elements independently at random from a set of size 2^n, with probability > ⅓ you will pick at least one element twice.

- Can think of computing H(random) as picking a random n-bit hash value/string

- By Birthday: After hashing $2^{n/2}$ random inputs, we're likely to find a collision

- If hash function has n-bit output, brute-force attack finds collision in $O(2^{n/2})$ time

- This is why cryptographic hash functions have 256-bit output
  ⇒ To defend against brute-force attack running in 2^{128} time.

Why it is called the “Birthday Paradox?”

- A group of 23 people are >50% likely to have at least one birthday in common



Intuition: Collision resistant hashes

- A hash function provides a fixed-length “fingerprint” for a sequence of bits
- Just as a fingerprint is a short unique (?) identifier for a person,
  the hash of a file is a short unique identifier of file contents
- Very convenient! If Alice and Bob each have a 10000 TB file and they want to check if their files are identical: just exchange 256 bit hash!
  - Hash value is always short, no matter how long the files are

Examples: Hash functions that are **NOT** collision resistant

$H(M)$:= Chop M into n-bit blocks. Output XOR of all blocks

Collision: $H(1^n \,||\, 1^n) = 1^n \oplus 1^n = 0^n = H(0^n)$

$H(M)$:= Output first n bits of M

Collision: $H(0^n \,||\, 0^n) = 0^n = H(0^n)$

Other hash functions that are NOT collision resistant:

- `crc32`, Python's `hash()`, SipHash, …
- Any function with < 256 bit output (via Birthday paradox)
  - Example: Hash(x) := AES.*E*("00…0", x) – AES has 128-bit block size!

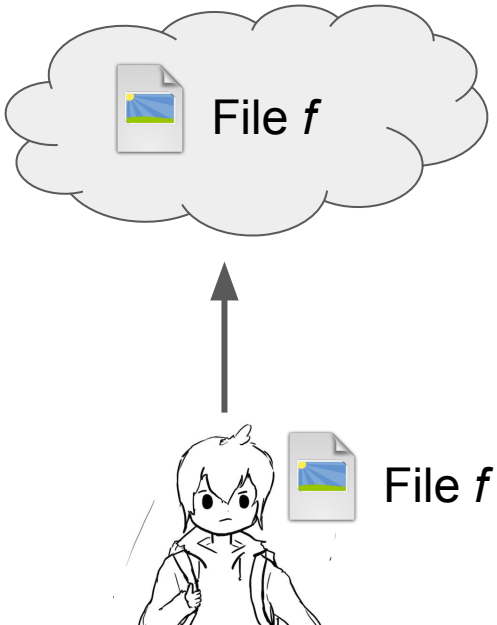
Examples: Hash functions that **ARE** collision resistant

(As far as we know)

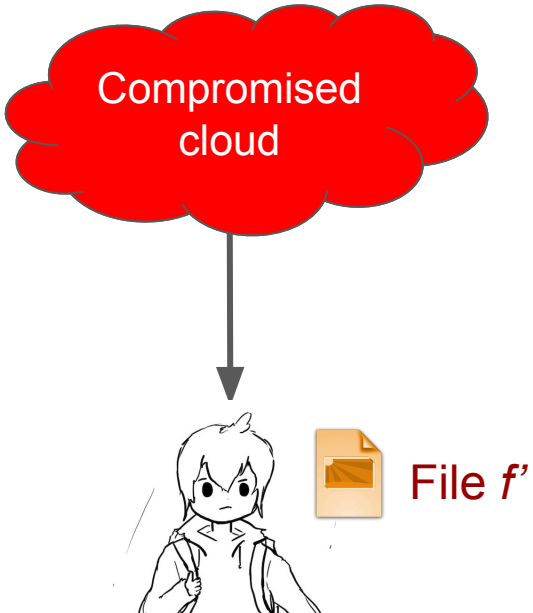
- ~~MD5~~
  - Output: 128 bits
  - Broken by 2^{64} brute-force attack, best attacks are much faster: 2^{34} time
- ~~SHA-1~~
  - Output: 160 bits
  - Security: Collision published in 2017
  - Was known to be weak before 2017, but still used sometimes
- SHA-2
  - Output: 256, 384, or 512 bits (sometimes labeled SHA-256, SHA-384, SHA-512)
  - Current standard, used everywhere
  - Modern CPUs have SHA2 hardware instructions – 3GB/s on my laptop
- SHA-3 (Keccak)
  - Output: 256, 384, or 512 bits
  - Designed as an alternative to SHA-2 family
  - Current standard, much less common to see in practice

Application of collision resistant hash: Authenticating a file

Today: You store a file *f* in the cloud



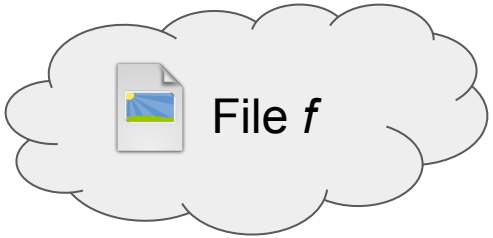
Tomorrow: You download the file.



Application of collision resistant hash:
Authenticating a file

Today: You store a file *f* in the cloud

+ Store a short digest locally



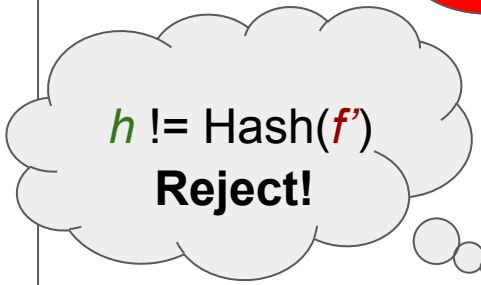


h ← Hash(*f*)

Tomorrow: You download the file.

+ Check that hash matches



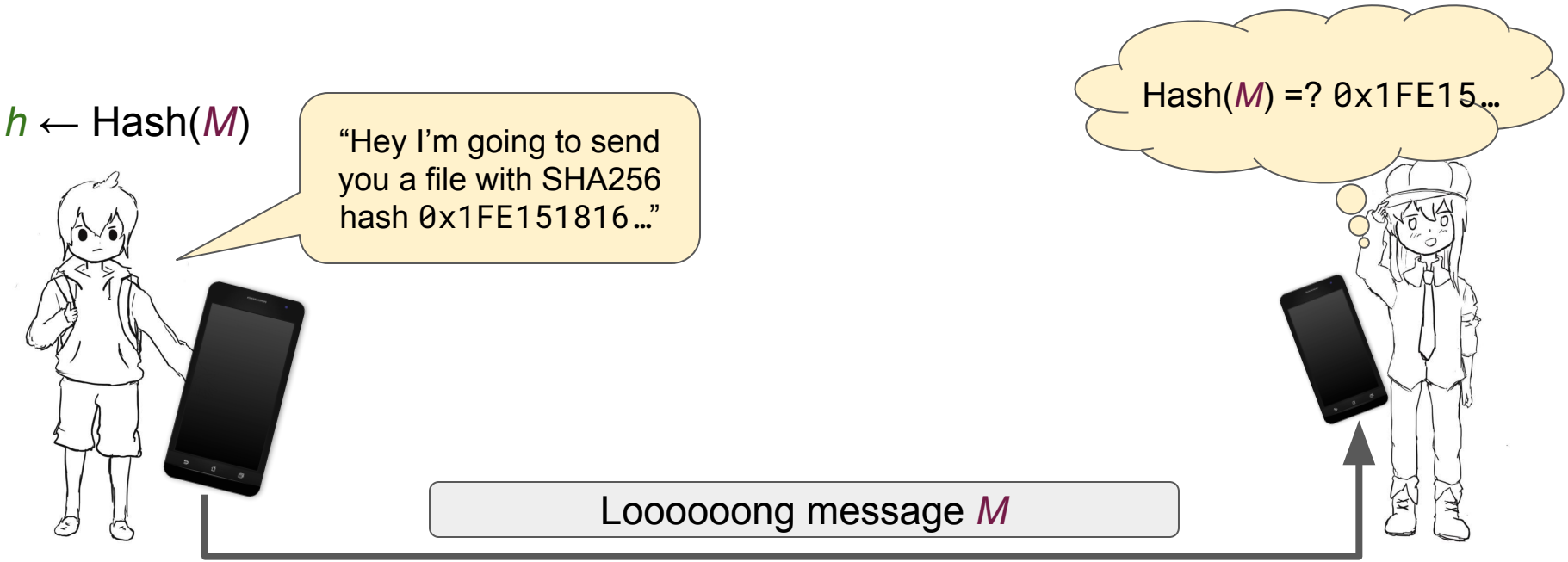




File *f'*

h

Application of collision resistant hash:
Authenticating a long message with a short **authenticated** string



Application of collision resistant hash:
Subresource integrity (SRI)

Mysite.com

```
<html>
<script
  src="https://sus.com/jquery.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5u…" />
```

sus.com



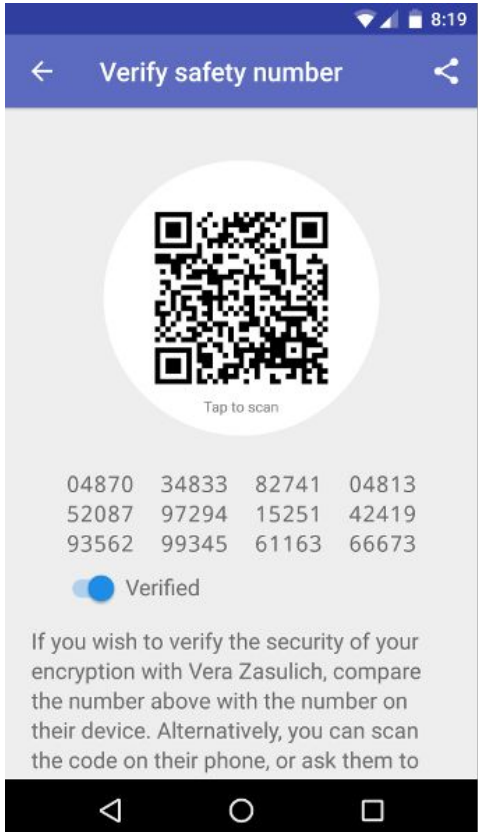


Hash() != oqV…

Reject!

Application of collision resistant hash: WhatsApp and Signal safety numbers

- Hash of both parties' public keys
  (will learn about public keys next week)
- Should be the same on both ends
  if there is no attacker in the middle



Application of collision resistant hash:
Validating software packages downloaded from mirror

Arch Linux Downloads

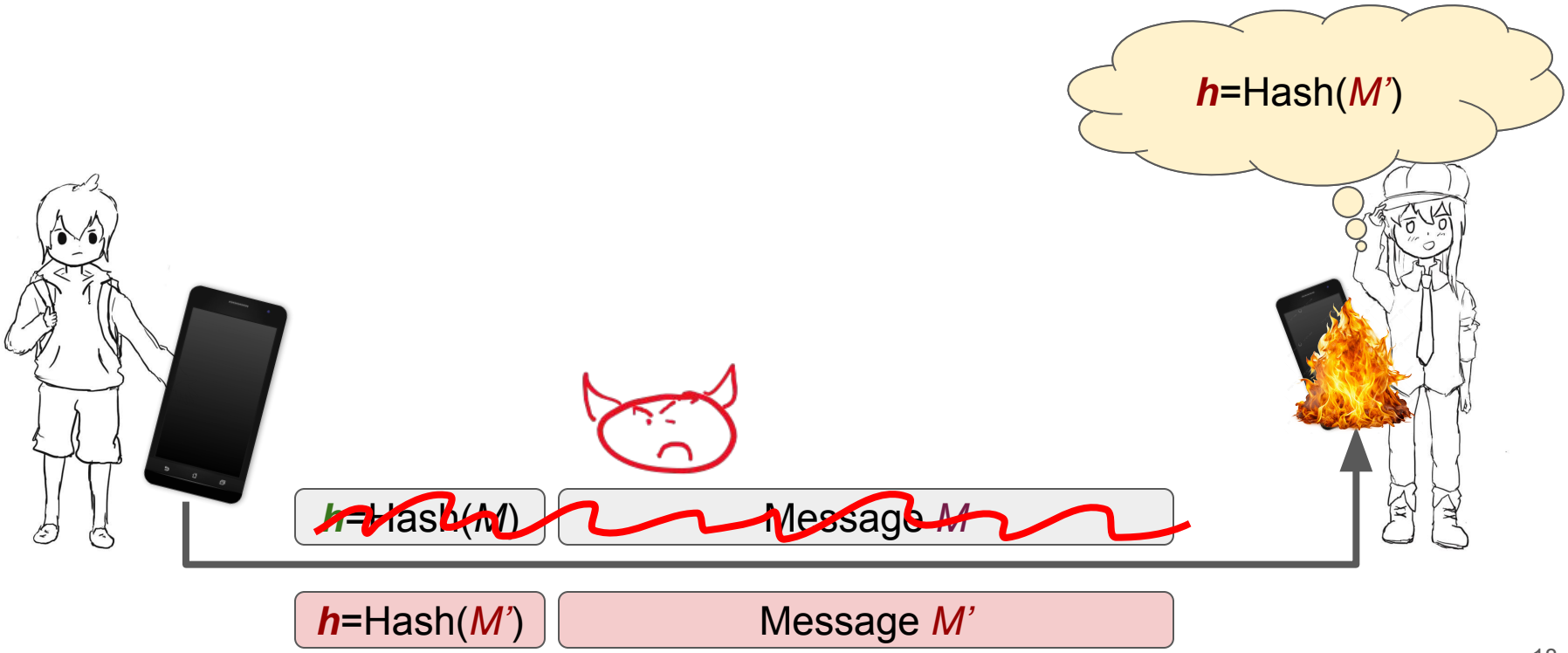
Checksums and signatures

File integrity checksums and PGP signatures for the latest releases can be found below:

- ISO
  - PGP signature
  - **PGP fingerprint:** 0x54449A5C
  - **SHA256:** c0ee0dab0a181c1d6e3d290a81ae9bc41c329ecaa00816ca7d62a685aeb8d972

BAD example:

Authenticating a long message with a short string



Lesson

If attacker can modify hash value, checking message against hash value does not protect integrity.

Hash function != Pseudorandom function

- People who have not taken CS161 may swap one crypto tool for another
  - This is dangerous

- Hash function output “looks random”-ish… can we use it as a PRF?
  - Salt and sugar look the same
  - Does not mean you can swap them in your cookie recipe
  - Type mismatch: PRF takes “(key, input)”. Hash takes only “input”

- **Bad idea:** $PRF_{Hash}(k, m) := Hash(k \,||\, m)$
  - Even if Hash is collision resistant and one-way, PRF_{Hash} is broken as a PRF!
  - “Length-extension attack” – See homework
  - Right way to build PRF from hash: “HMAC”

Building hash functions

- We cannot prove that any function is collision resistant.
  - Similar to situation with PRFs and block ciphers
- Instead, we assume that certain standard hash functions are collision resistant
  - We base these assumptions on many years of cryptanalysis
- As with any crypto tool: Don't attempt to cook your own!
  - Standard ones are fast and safe

Common paradigm:

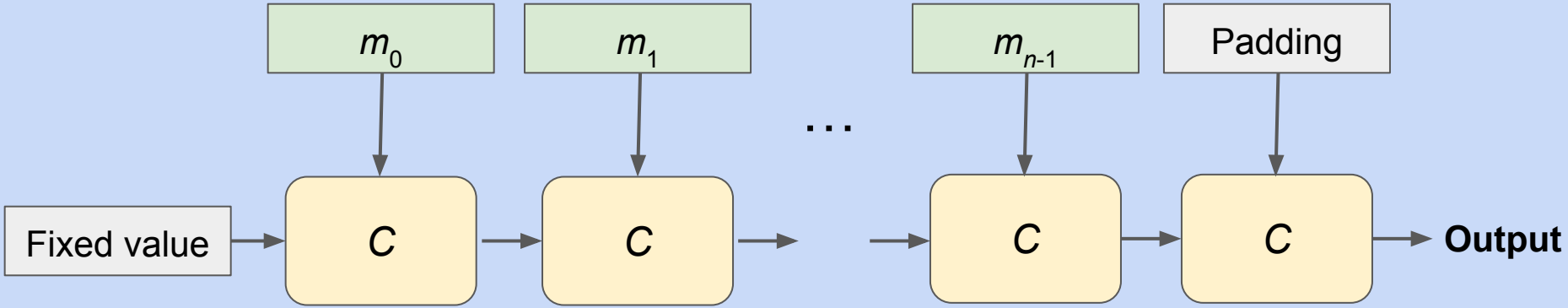
– Build a CRHF on short inputs ← Can't prove is CRHF; Rely on cryptanalysis

– Extend CRHF to handle long inputs ← Can prove big function is CRHF if small one is CRHF

Merkle-Damgard construction:

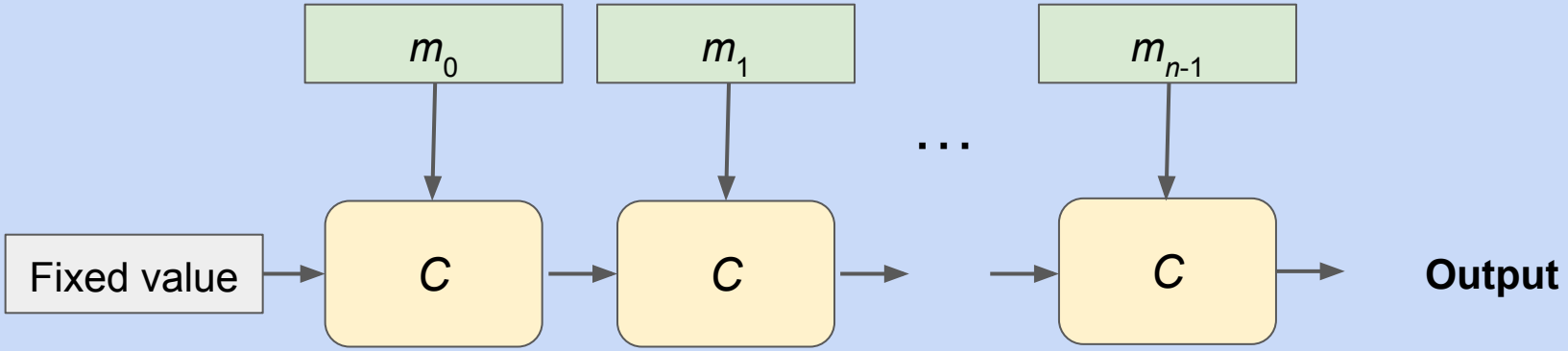
How to build a big CRHF from a small CRHF

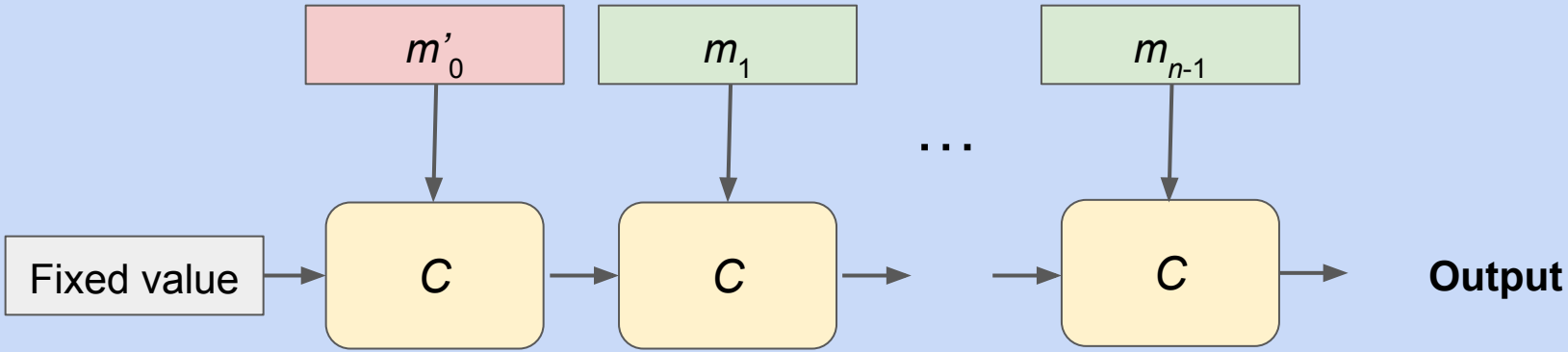
- Assume we have a function $C : \{0,1\}^{2n} \rightarrow \{0,1\}^n$ that we believe to be CRHF
- Then the following gives a CRHF on arbitrary-length inputs



Merkle-Damgard construction:

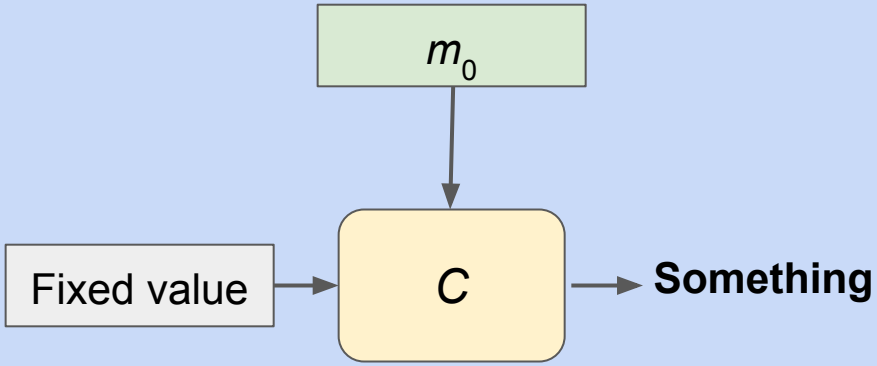
How to build a big CRHF from a small CRHF

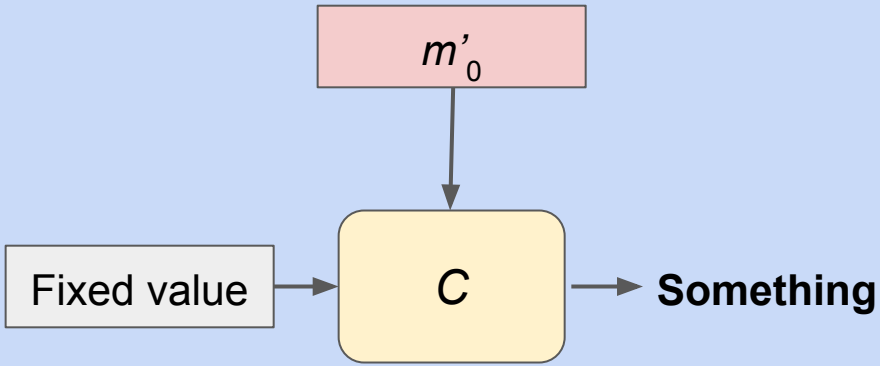




Merkle-Damgard construction:

How to build a big CRHF from a small CRHF





We found a collision in C:
$C(\text{Fixed}, m_0) = C(\text{Fixed}, m'_0)$

Definition: One-way function (OWF)

An efficiently computable function $H : \{0,1\}^n \rightarrow \{0,1\}^n$ is a **one-way function** if:

- Informally: "It is hard to invert H on random inputs."
- Formally: For all efficient algorithms A,
  - $\Pr[H(x') = H(x) : x \leftarrow_R \{0,1\}^n, x' \leftarrow A(H(x))] <=$ "negligible"

All CRHFs are also one-way functions
– All one-way are **NOT** CRHFs

Question:

- Let $H : \{0,1\}^n \rightarrow \{0,1\}^n$ be a one-way function.
- Alice publishes $h = H($`alice_birthday`$)$.
- Is it hard for an attacker to recover `alice_birthday` given only h?

Application of one-way hash: Password storage

- On Linux-like OSes, a **hash** of every user's password is stored in file `/etc/shadow`
- When you type in a password *pass*:
  - OS computes *h'* = Hash(*pass*)
  - OS fetches true hash *h* from `/etc/shadow`
  - If *h* = *h'*, OS allows user to log in

- What happens if attacker steals `/etc/shadow`?
  - Needs to invert the hashes ("password cracking")
  - Easy if your password is not very strong—just guess
  - Not easy if you use a strong password

Hashes: Summary

- Deterministic function from arbitrary-length input to fixed-length output

- Security properties
  - **Collision resistant:** It is hard to find $x' \neq x$ such that $H(x) = H(x')$.
  - **One way:** Given a $y \leftarrow H(\text{random})$, it is hard to find an x such that $H(x) = y$.

- The magic of collision resistant hashes:
  You can authenticate a very long message by authenticating a 256-bit string

- Examples: SHA2, SHA3 (Keccak), …

Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) ✅ Hash functions (e.g., SHA2)	– Discrete-log problem